Cloud Platforms Assignment #4

By: Baran Erdogan

Step 1: Fork the Repository

The steps in the instructions page were followed. In figure #1, the forked repository can be seen in my profile.

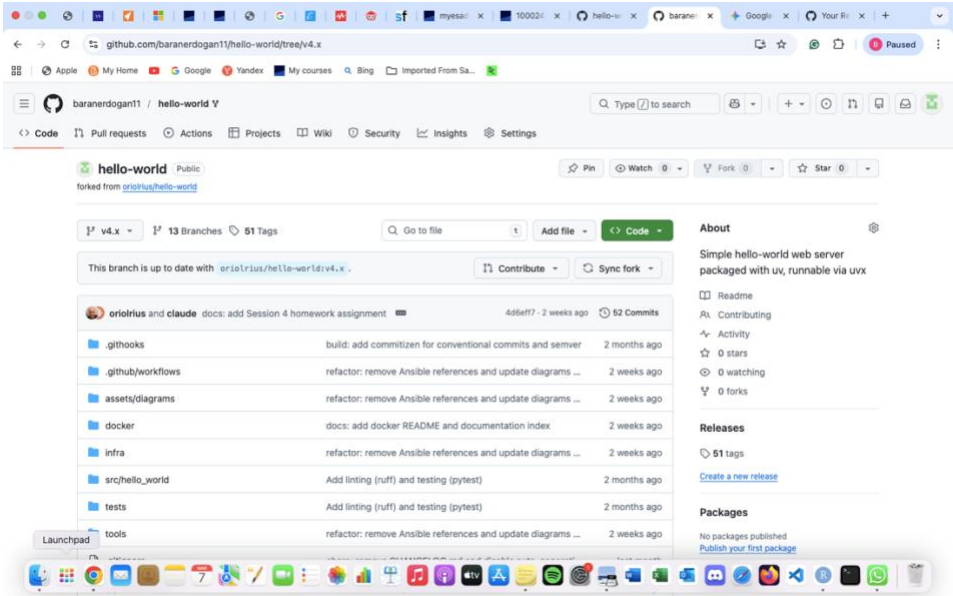

Figure 1: Forked Repository

Step 2: Configure AWS Credentials

AWS credentials were obtained from the AWS innovation sandbox webpage. After, they were added as repository secrets to the GitHub repository.

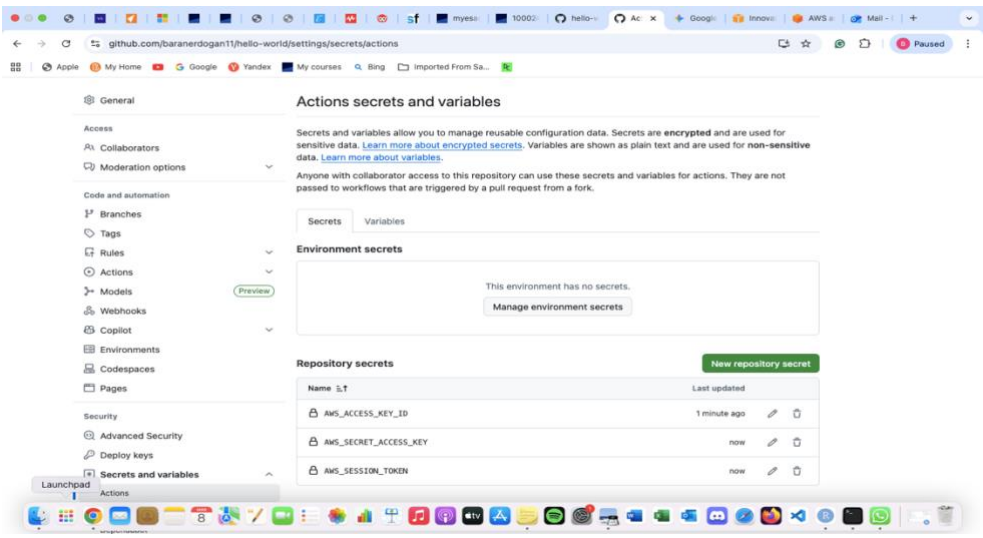

Figure 2: AWS Credentials as Secrets

Step 3: Run the GitHub Actions Workflow

Instructions were followed, and the workflow was executed. It took 6 minutes and 22 seconds to get the green light from every job. The workflow can be seen in figure 3.

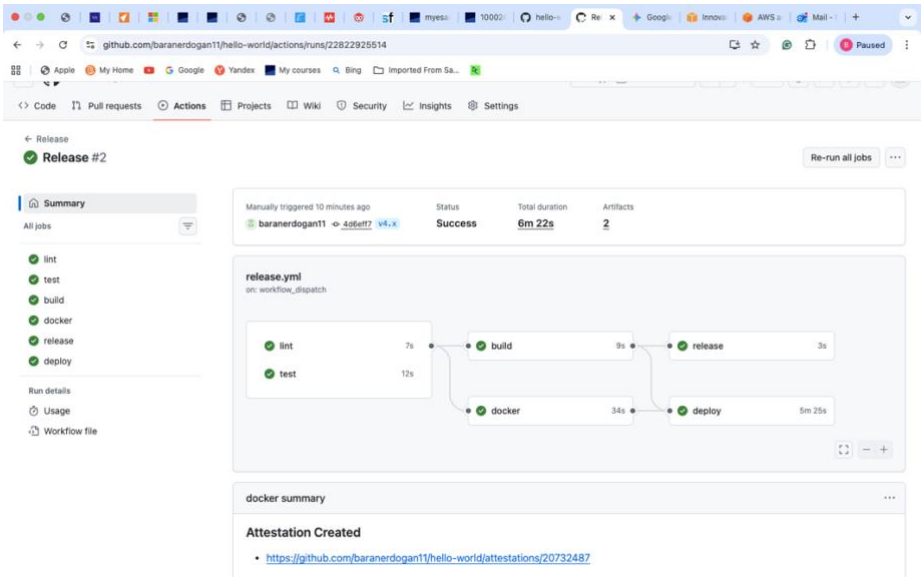

Figure 3: Executed Workflow

In the Packages section, hello-world container image was published.

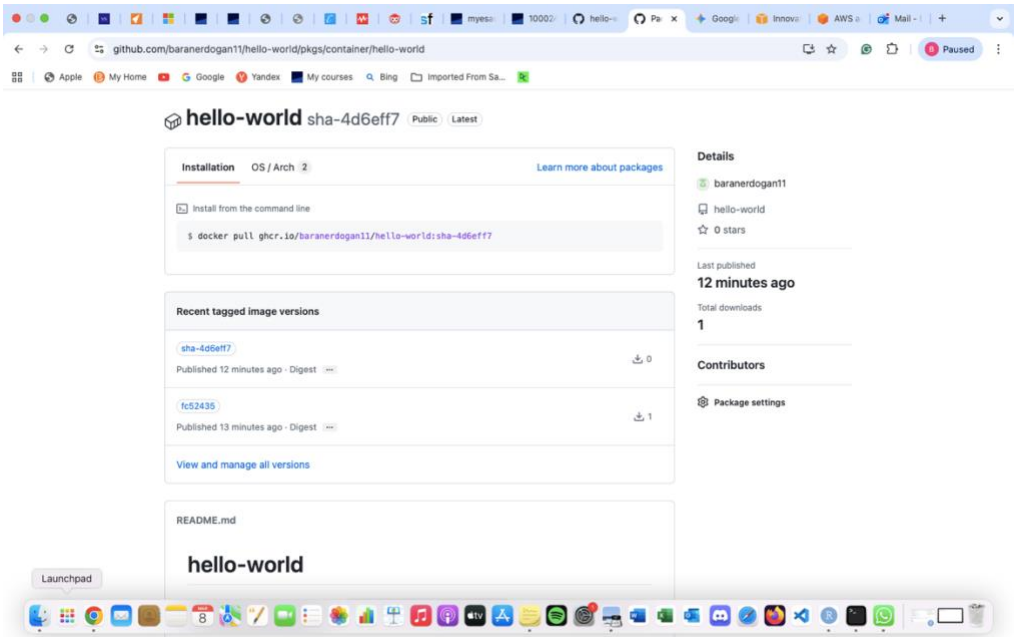

Figure 4: hello-world Container Image

After this step, hello-world AWS CloudFormation Stack was checked to see if the hello-world stack was created, and visible. The confirmation can be seen in figure 5.

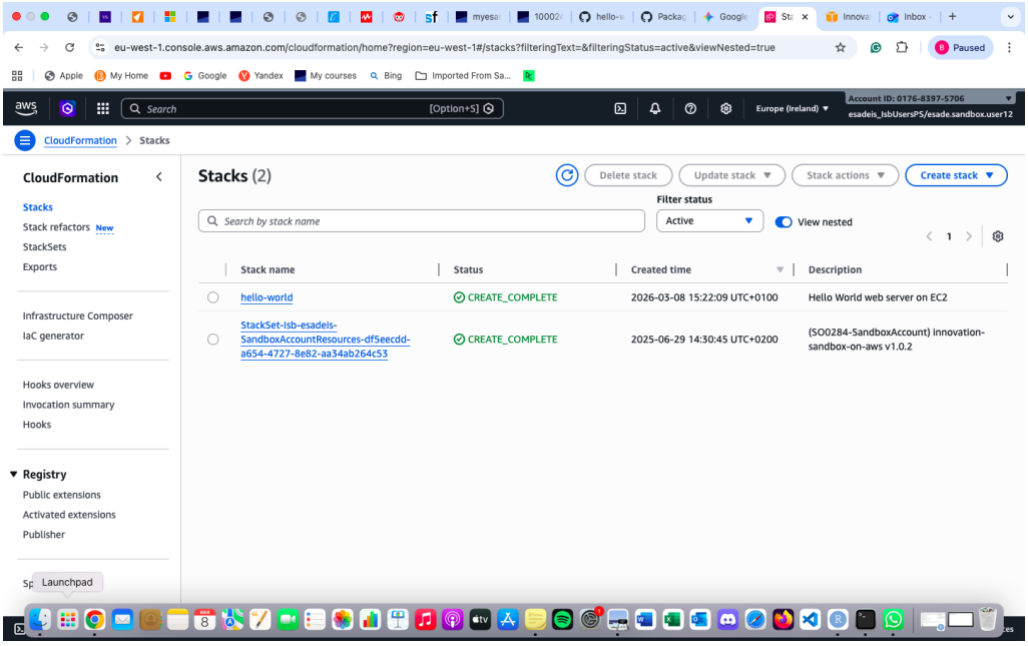

Figure 5: hello-world AWS CloudFormation Stack

Furthermore, in the AWS console, EC2 Instances part was checked to confirm if hello-world EC2 instance was created, and running. The proof is visible in figure 6.

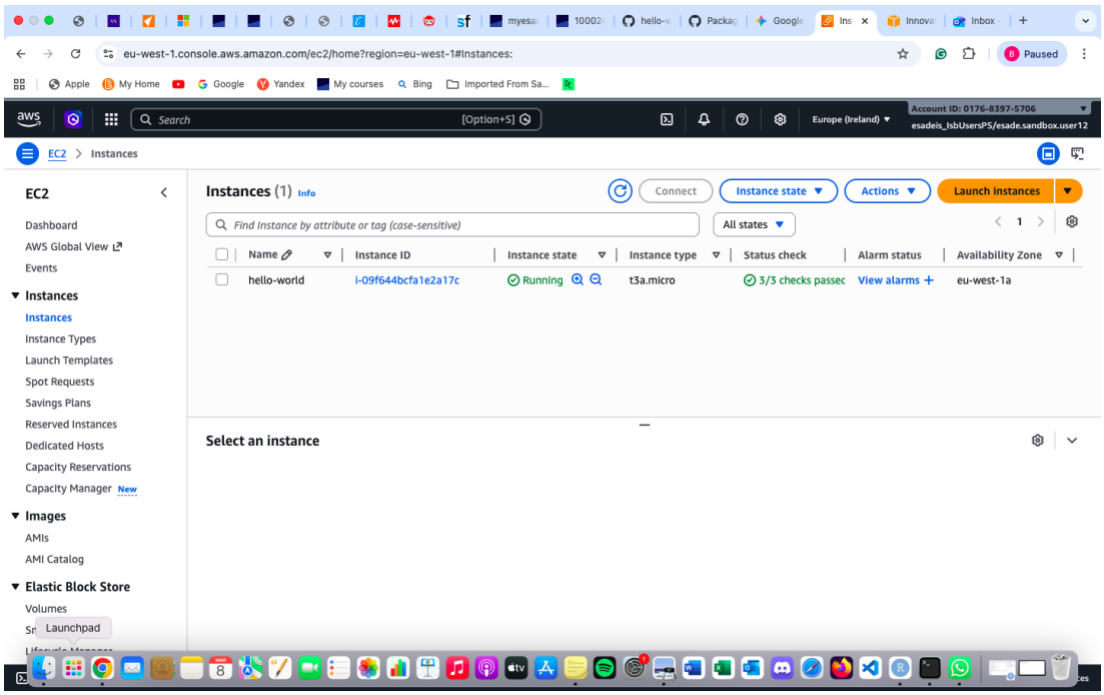

Figure 6: hello-world EC2 Instance

Step 4: Document the CI/CD Process

How did you trigger the workflow?

In GitHub, in the Actions tab, “Release” section was chosen, and “Run Workflow” button was hit. As mentioned, the whole “green light” process took 6 minutes and 22 seconds. It was ensured that v4.x branch was chosen.

What is the purpose of each job?

Lint: Uses Ruff to analyze the code for stylistic errors or "smelly" code without actually running it.

Test: Executes unit tests using Pytest to ensure the application logic is correct and hasn't regressed.

Build: Compiles the Python code into a distributable package (a .whl wheel file) using uv.release

Release: Automates the creation of a GitHub Release and attaches the built artifacts (the wheel file) to it.

Docker: Packages the application and its environment into a Docker image and pushes it to the GitHub Container Registry (GHCR).

Deploy: Uses AWS CloudFormation to provision an EC2 instance and run the Docker container on AWS infrastructure.

How do the jobs depend on each other?

The workflow uses a mix of parallel and sequential execution to balance speed with safety:

- **Parallel Execution:** The lint and test jobs run at the same time. This is efficient because they don't depend on each other; if the code is ugly (lint fails) but works (test passes), we still want to know both results immediately.

- **Sequential Execution:** The build, docker, and deploy jobs must run in order.
    - Build depends on lint and test passing.
    - Docker and release depend on build.
    - Deploy depends on docker.

What credentials are needed and why?

AWS credentials and GitHub token were needed. AWS credentials were needed to authenticate and access to the AWS Innovation Sandbox account. GitHub token was needed to provide the workflow with permissions to upload the Docker image to GHCR and create the GitHub Release within the same repository.

Step 5: Add the whoami Service

Once again, instructions were followed to implement the whoami YAML code to the file. Modified code can be seen in figure 7.

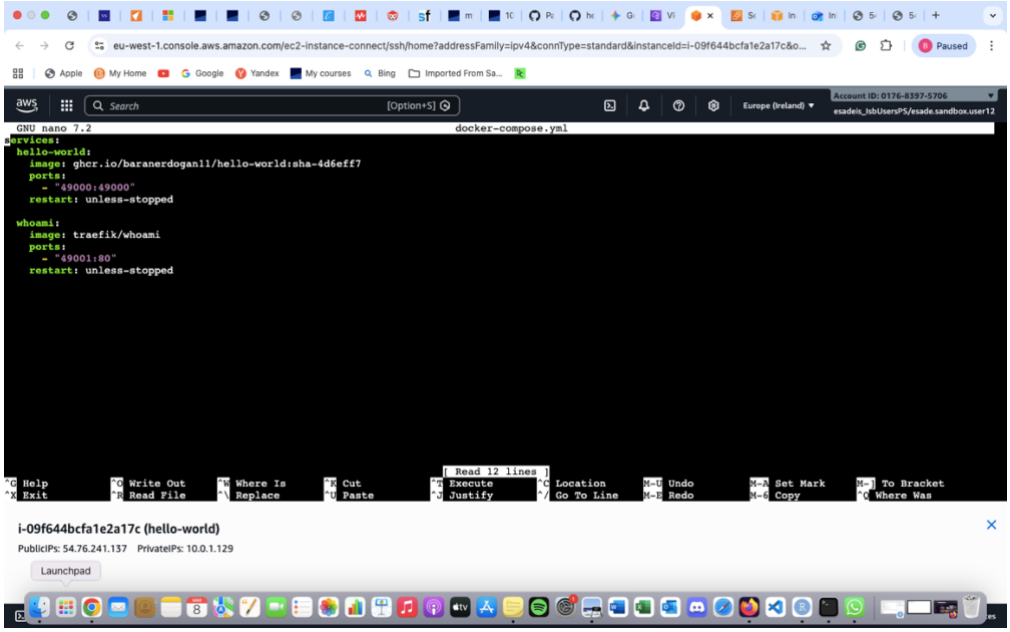

Figure 7: whoami Service Code

Afterwards, both containers were confirmed to be running.

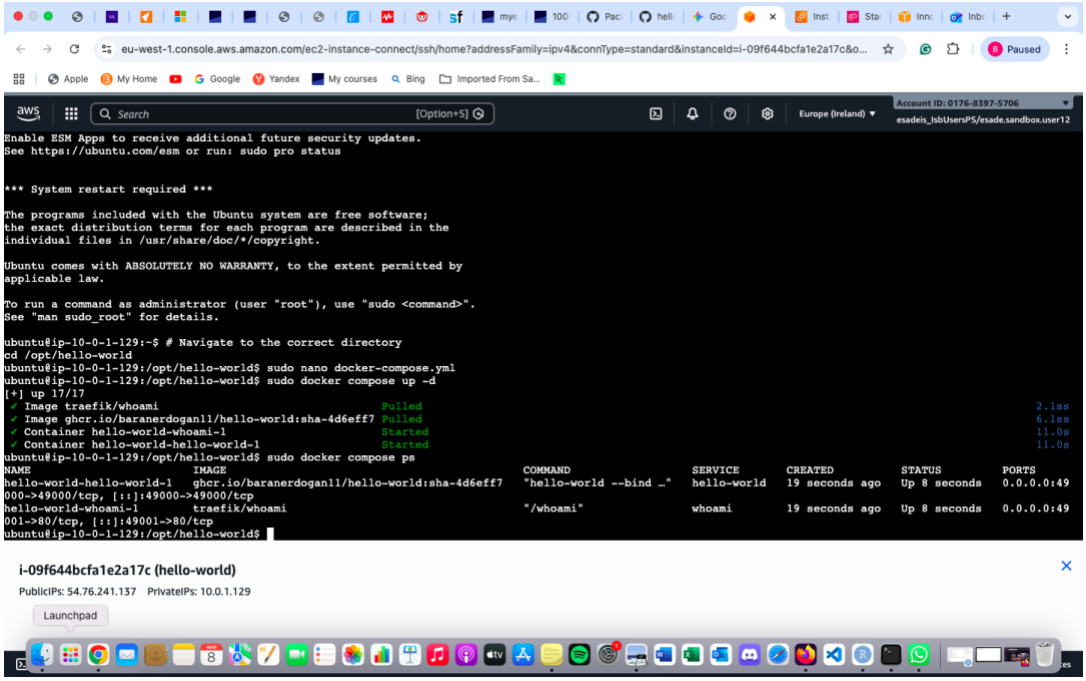

Figure 8: Running Containers

Step 6: Verify Both Services

Finally, both services were confirmed to be running. Figures 9 and 10 confirm the services, and public IP addresses.

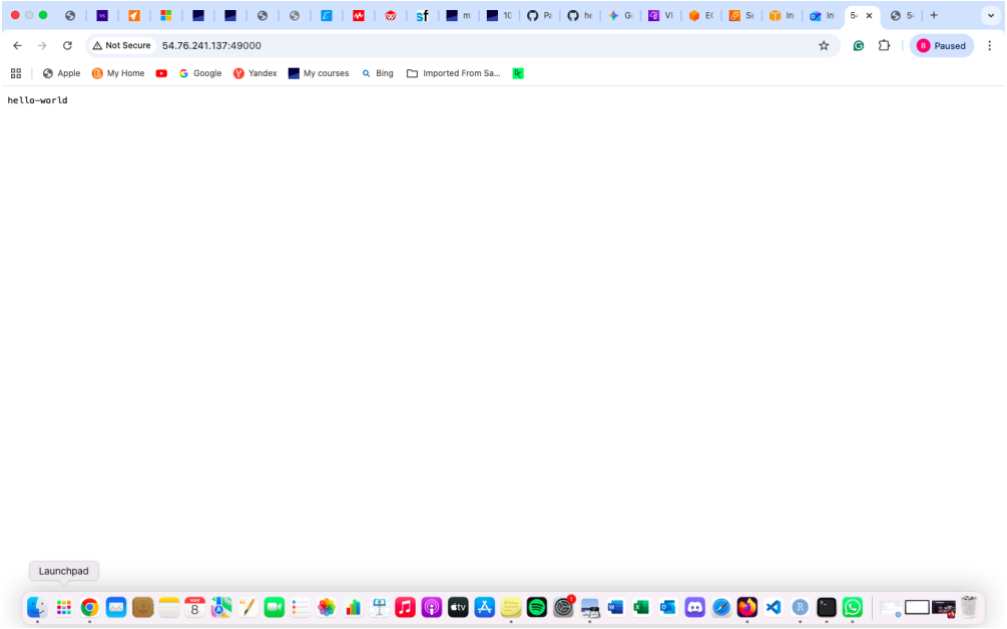

Figure 9: hello-world Service

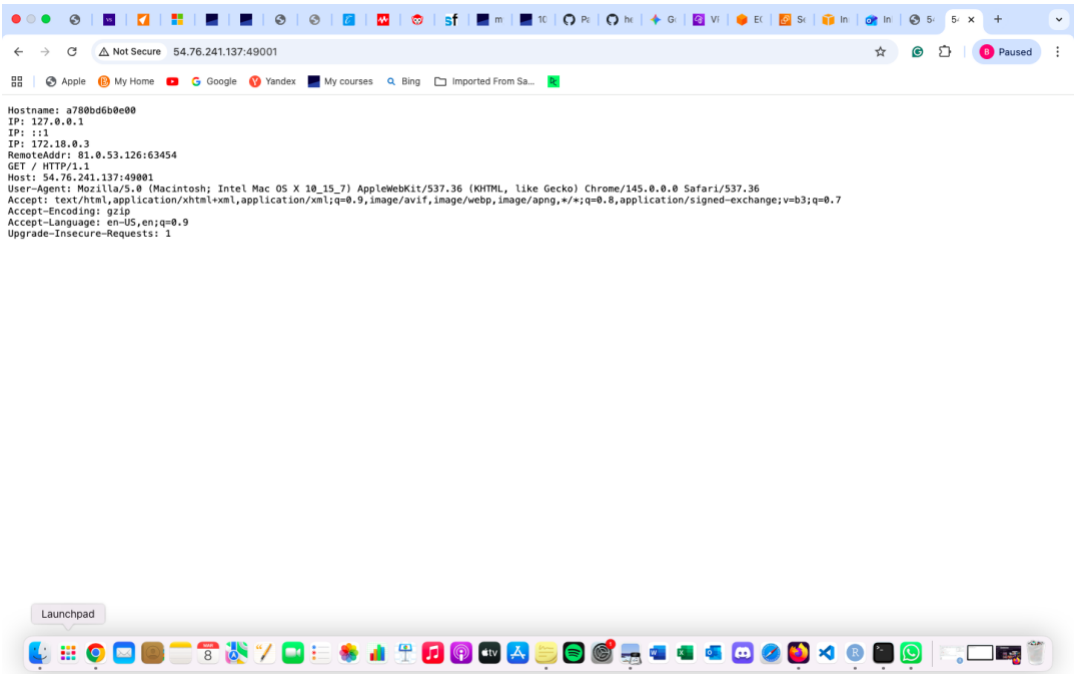

Figure 10: whoami Service